```
    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 3

1. The constructor is called **widgit()** and the destructor is called ~**widgit()**.

2. The constructor function is called when an object is created (that is, when an object comes into existence). The destructor is called when an object is destroyed.

3. ```
   class Mars : public planet
   {
       // ...
   };
   ```

4. You can expand a function in line either by preceding its definition with the **inline** specifier or by including its definition within a class declaration.

5. An in-line function must be defined before it is first used. Other common restrictions include the following: It cannot contain any loops. It must not be recursive. It cannot contain a **goto** or a **switch** statement. It cannot contain any **static** variables.

6. ```
   sample ob(100, 'X');
   ```

3.1 EXERCISES

1. The assignment statement **x = y** is wrong because **cl1** and **cl2** are two different types of classes, and objects of differing class types cannot be assigned.

2. ```
   #include <iostream>
   using namespace std;

   #define SIZE 100

   class q_type
   {
       int queue[SIZE]; // holds the queue
       int head, tail; // indices of head and tail
   public:
       q_type(); // constructor
       void q(int num); // store
       int deq(); // retrieve
   };

   // Constructor
   q_type::q_type()
   {
       head = tail = 0;
   }

   // Put value on the queue.
   ```
   ```

```
void q_type::q(int num)
{
    if(tail+1==head || (tail+1==SIZE && !head))
    {
        cout << "Queue is full\n";
        return;
    }
    tail++;
    if(tail==SIZE)
        tail = 0; // cycle around
    queue[tail] = num;
}

// Remove a value from a queue.
int q_type::deq()
{
    if(head == tail)
    {
        cout << "Queue is empty\n";
        return 0; // or some other error indicator
    }
    head++;
    if(head==SIZE)
        head = 0; // cycle around
    return queue[head];
}

int main()
{
    q_type q1, q2;
    int i;

    for(i=1; i<=10; i++)
        q1.q(i);

    // assign one queue to another
    q2 = q1;

    // show that both have the same contents
    for(i=1; i<=10; i++)
        cout << "Dequeue 1: " << q1.deq() << "\n";

    for(i=1; i<=10; i++)
        cout << "Dequeue 2: " << q2.deq() << "\n";

    return 0;
}
```

3. If memory to hold a queue is dynamically allocated, assigning one queue to another causes the dynamic memory allocated to the queue on the left side of the assignment statement to be lost and the memory allocated to the queue on the right side to be freed twice when

the objects are destroyed. Either of these two conditions is an unacceptable error.

3.2 EXERCISES

1.
```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
    stack(); // constructor
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack
stack::stack()
{
    cout << "Constructing a stack\n";
    tos = 0;
}

// Push a character.
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return [tos];
}
```

```
void showstack(stack o);

int main()
{
    stack s1;
    int i;

    s1.push('a');
    s1.push('b');
    s1.push('c');

    showstack(s1);

    // s1 in main is still existent
    cout << "s1 stack still contains this: \n";
    for(i=0; i<3; i++)
        cout << s1.pop() << "\n";

    return 0;
}

// Display the contents of a stack.
void showstack(stack o)
{
    char c;

    // when this statement ends, the o stack is empty
    while(c=o.pop())
        cout << c << '\n';
}
```

This program displays the following:

```
Constructing a stack
c
b
a
Stack is empty
s1 stack still contains this:
c
b
a
```

2. The memory used to hold the integer pointed to by **p** in object **o** that is used to call **neg()** is freed when the copy of **o** is destroyed when **neg()** terminates, even though this memory is still needed by **o** inside **main()**.

3.3 EXERCISES

1. ```
   #include <iostream>
   using namespace std;

   class who
   {
       char name;
   public:
       who(char c)
       {
           name = c;
           cout << "Constructing who #";
           cout << name << '\n';
       }
       ~who() { cout << "Destructing who #" << name << "\n"; }
   };

   who makewho()
   {
       who temp('B');
       return temp;
   }

   int main()
   {
       who ob('A');

       makewho();

       return 0;
   }
   ```

2. There are several situations in which it would be improper to return an object. Here is one: if an object opens a disk file when it is created and closes that file when it is destroyed, if that object is returned from a function, the fill will be closed when the temporary object is destroyed.

3.4 EXERCISES

1. ```
   #include <iostream>
   using namespace std;

   class pr2; // forward declaration

   class pr1
   {
       int printing;
       // ...
   public:
       pr1() { printing = 0; }
   ```

```
    void set_print(int status) { printing = status; }
    // ...
    friend int inuse(pr1 o1, pr2 o2);
};

class pr2
{
    int printing;
    // ...
public:
    pr2() { printing = 0; }
    void set_print(int status) { printing = status; }
    // ...
    friend int inuse(pr1 o1, pr2 o2);
};

// Return true if printer is in use.
int inuse(pr1 o1, pr2 o2)
{
    if(o1.printing || o2.printing)
        return 1;
    else
        return 0;
}

int main()
{
    pr1 p1;
    pr2 p2;

    if(!inuse(p1, p2))
        cout << "Printer idle\n";

    cout << "Setting p1 to printing...\n";
    p1.set_print(1);
    if(inuse(p1, p2))
        cout << "Now printer in use.\n";

    cout << "Turn off p1...\n";
    p1.set_print(0);
    if(!inuse(p1, p2))
        cout << "Printer idle\n";

    cout << "Turn on p2...\n";
    p2.set_print(1);
    if(inuse(p1, p2))
        cout << "Now printer in use.\n";

    return 0;
}
```

MASTERY SKILLS CHECK: Chapter 3

1. For one object to be assigned to another, both must be of the same class type.

2. The trouble with the assignment of **ob1** to **ob2** is that the memory pointed to by **ob2**'s initial value of **p** is now lost because this value is overwritten by the assignment. This memory thus becomes impossible to free, and the memory pointed to by **ob1**'s **p** is freed twice when it is destroyed-possibly causing damage to the dynamic allocation system.

3. ```
   int light(planet p)
   {
       return p.get_miles() / 186000;
   }
   ```

4. Yes.

5. ```
   // Load a stack with the alphabet.
   #include <iostream>
   using namespace std;

   #define SIZE 27

   // Declare a stack class for characters
   class stack
   {
       char stck[SIZE]; // holds the stack
       int tos; // index of top of stack
   public:
       stack(); // constructor
       void push(char ch); // push character on stack
       char pop(); // pop character from stack
   };

   // Initialize the stack
   stack::stack()
   {
       cout << "Constructing a stack\n";
       tos = 0;
   }

   // Push a character.
   void stack::push(char ch)
   {
       if(tos==SIZE)
       {
           cout << "Stack is full\n";
           return;
       }
       stck[tos] = ch;
       tos++;
   }
   ```

```
// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

void showstack(stack o);
stack loadstack();

int main()
{
    stack s1;

    s1 = loadstack();
    showstack(s1);

    return 0;
}

// Display the contents of a stack.
void showstack(stack o)
{
    char c;

    // when this statement ends, the o stack is empty
    while(c=o.pop())
        cout << c << '\n';
}

// Load a stack with the letters of the alphabet.
stack loadstack()
{
    stack t;
    char c;
    for(c = 'a'; c <= 'z'; c++)
        t.push(c);

    return t;
}
```

6. When passing an object to a function or returning an object from a function, temporary copies of the object are created that will be destroyed when the function terminates. When a temporary copy of an object is destroyed, the destructor function might destroy something that is needed elsewhere in the program.

7. A friend is a nonmember function that is granted access to the private members of the

class for which it is a friend.

CUMULATIVE SKILLS CHECK: Chapter 3

1.
```
// Load a stack with the alphabet.
#include <iostream>
#include <cctype>
using namespace std;

#define SIZE 27

// Declare a stack class for characters
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
    stack(); // constructor
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack
stack::stack()
{
    cout << "Constructing a stack\n";
    tos = 0;
}

// Push a character.
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
```

```
}

void showstack(stack o);
stack loadstack();
stack loadstack(int upper);

int main()
{
    stack s1, s2, s3;

    s1 = loadstack();
    showstack(s1);

    // get uppercase letters
    s2 = loadstack(1);
    showstack(s2);

    // use lowercase letters
    s3 = loadstack(0);
    showstack(s3);

    return 0;
}

// Display the contents of a stack.
void showstack(stack o)
{
    char c;

    // when this statement ends, the o stack is empty
    while(c=o.pop())
        cout << c << '\n';
}

// Load a stack with the letters of the alphabet.
stack loadstack()
{
    stack t;
    char c;
    for(c = 'a'; c <= 'z'; c++)
        t.push(c);

    return t;
}

/*
   Load a stack with the letters of the alphabet. Uppercase
   letters if upper if 1; lowercase otherwise.
*/
stack loadstack(int upper)
{
```

```
    stack t;
    char c;

    if(upper)
        c = 'A';
    else
        c = 'a';

    for(; toupper(c) <= 'Z'; c++)
        t.push(c);
    return t;
}
```

2.
```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr);
    ~strtype();
    void show();
    friend char *get_string(strtype *ob);
};

strtype::strtype(char *ptr)
{
    len = strlen(ptr);
    p = (char *) malloc(len+1);
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    strcpy(p, ptr);
}

strtype::~strtype()
{
    cout << "Freeing p\n";
    free(p);
}

void strtype::show()
{
    cout << p << " - lengthj: " << len;
    cout << '\n';
```

```
}

char *get_string(strtype *ob)
{
    return ob->p;
}

int main()
{
    strtype s1("This is a test.");
    char *s;

    s1.show();

    // get pointer to string
    s = get_string(&s1);
    cout << "Here is string contained in s1: ";
    cout << s << "\n";

    return 0;
}
```

3. The outcome of the experiment is as follows: Yes, data from the base class is also copied when an object of a derived class is assigned to another object of the same derived class. Here is a program that demonstrates this fact:

```
#include <iostream>
using namespace std;

class base
{
    int a;
public:
    void load_a(int n) { a = n; }
    int get_a() { return a; }
};

class derived : public base
{
    int b;
public:
    void load_b(int n) { b = n; }
    int get_b() { return b; }
};

int main()
{
    derived ob1, ob2;

    ob1.load_a(5);
    ob1.load_b(10);
```

```
    // assign ob1 to ob2
    ob2 = ob1;

    cout << "Here is ob1's a and b: ";
    cout << ob1.get_a() << " " << ob1.get_b() << "\n";

    cout << "Here is ob2's a and b: ";
    cout << ob2.get_a() << " " << ob2.get_b() << "\n";

    /*
       As you can probably guess, the output is the same for
       each object.
    */

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 4

1. When one object is assigned to another of the same type, the current values of all data members of the object on the right are assigned to the corresponding data members on the left.

2. Trouble can occur when one object is assigned to another if that assignment overwrites important data already existing in the target object. For example, a pointer to dynamic memory or to an open file can be overwritten and, therefore, lost.

3. When an object is passed to a function, a copy is made. However, the copy's destructor is called when the object is destroyed by the termination of the function.

4. The violation of the separation between an argument and its copy when passed to a parameter can be caused by several destructor, that memory will also be lost to the argument. In general, if the destructor function destroys anything that the original argument requires, damage to the argument will occur.

5.
```
#include <iostream>
using namespace std;

class summation
{
    int num ;
    long sum ; // summation of num
public:
    void set_sum ( int n);
    void show_sum ()
    {
        cout << num << " summed is " << sum << "\n";
    }
};

void summation :: set_sum (int n)
{
```